INSTITUTO FEDERAL
São Paulo
Campus
São Paulo

SPOCPGR

Profº João Temochko

O que é Rasterização

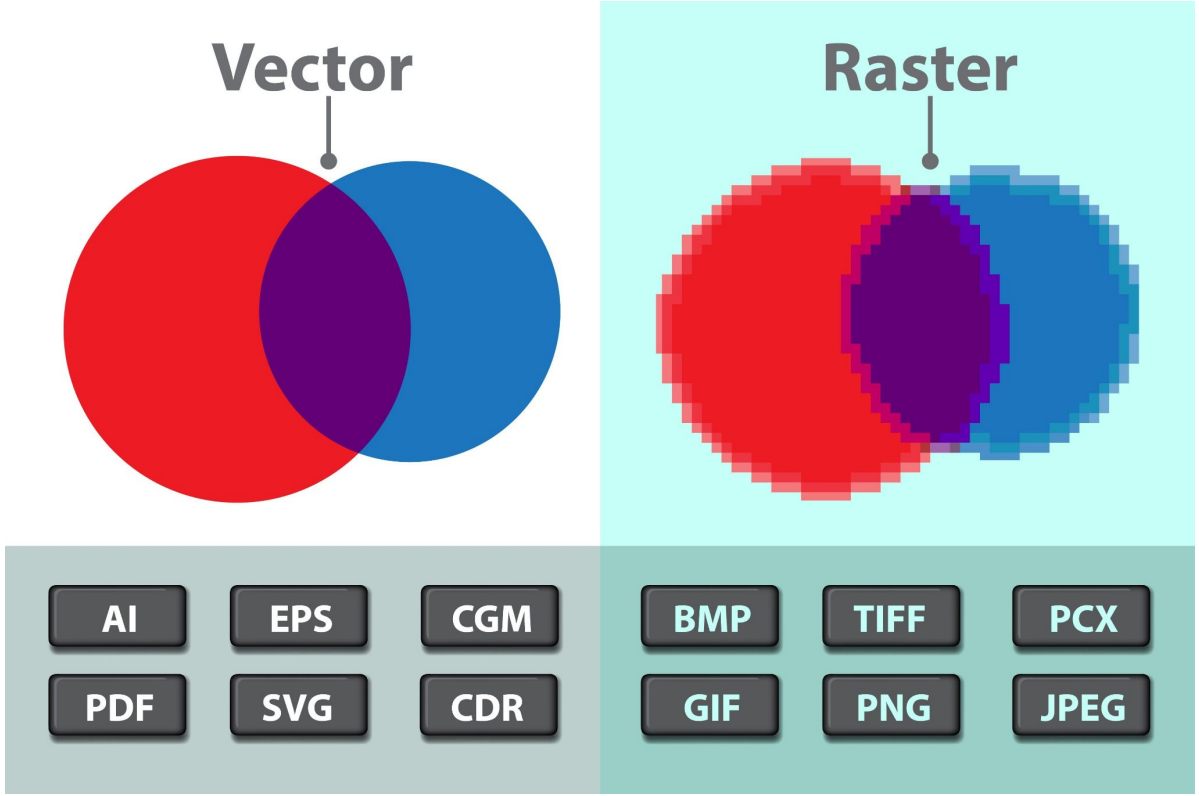
- É o processo de converter representações matemáticas (vetores) em uma matriz de pixels (raster).

- O monitor é uma grade finita, cheia de pixels que formam uma imagem; retas matemáticas têm infinitos pontos.

- Serrilhado (Aliasing): O artefato visual inevitável dessa conversão.

INSTITUTO FEDERAL
São Paulo
Campus São Paulo

O que é Rasterização



A Equação da Reta

- Geometria analítica: **y = mx + b**.

- **m**: Coeficiente angular (inclinação).

- **b**: Interseção em Y.

- Por que não usamos essa equação diretamente em um loop for? (R: Custo computacional de multiplicações e pontos flutuantes).

A Equação da Reta

- Simples para calcular, mas falha em retas verticais (onde a inclinação é infinita).

- Exemplo:
  - Como desenhar: Para cada pixel, se, x = 0, y = 1 (ponto (0,1)). Se x = 1, y = 3, (ponto (1,3)).

- Uso em CG: Facilita a plotagem direta de gráficos e rastreamento de raios (ray tracing) simples.

O Algoritmo DDA (Digital Differential Analyzer)

- Usa a diferença entre os pontos (Δx e Δy) para calcular incrementos passo a passo.

- Se a reta é mais horizontal ($|\Delta x| > |\Delta y|$), incrementamos X em 1 e calculamos o Y correspondente.

- Se a reta é mais horizontal ($|\Delta x| > |\Delta y|$), incrementamos X em 1 e calculamos o Y correspondente.

INSTITUTO FEDERAL
São Paulo
Campus São Paulo

Algoritmo DDA (Desenhando Pixel a Pixel)

```
#include <GL/glut.h>
#include <cmath>

// Função para desenhar um único pixel na tela
void desenhaPixel(int x, int y) {
    glBegin(GL_POINTS);
        glVertex2i(x, y);
    glEnd();
}

void algoritmoDDA(float x1, float y1, float x2, float y2) {
    float dx = x2 - x1;
    float dy = y2 - y1;

    // Identifica o maior incremento para determinar o número de passos
    int passos = std::max(std::abs(dx), std::abs(dy));

    // Calcula o incremento em X e Y para cada passo (aqui entra o float!)
    float incX = dx / (float)passos;
    float incY = dy / (float)passos;

    float x = x1;
    float y = y1;

    for (int i = 0; i <= passos; i++) {
        // Arredonda os valores float para o inteiro mais próximo e desenha
        desenhaPixel(std::round(x), std::round(y));
        x += incX;
        y += incY;
    }
}

// Dentro da sua função de display (glutDisplayFunc):
// glColor3f(1.0, 0.0, 0.0); // Cor vermelha
// algoritmoDDA(20.0, 20.0, 150.0, 80.0);
```

Prós e Contras do DDA

- **Vantagem:** Mais rápido que usar a equação da reta diretamente (usa adição em vez de multiplicação no loop).

- **Desvantagem:** Ainda depende de variáveis de ponto flutuante (float/double) e arredondamentos (round()), o que custa ciclos de processamento.

A Solução de Jack Bresenham

- Criado na IBM em 1962.

- Otimização extrema: Utiliza apenas aritmética inteira (somas, subtrações e deslocamentos de bits/multiplicação por 2).

- Ideal para implementação direta em hardware/microcontroladores.

Bresenham para Retas - A Variável de Decisão

- Em vez de calcular o Y exato, o algoritmo usa uma "variável de decisão" (erro acumulado) para decidir se o próximo pixel deve subir no eixo Y ou continuar reto.

- Atualização iterativa do erro a cada passo no eixo X principal.

Bresenham para Circunferências

- Aproveitando a simetria: Um círculo tem simetria de 8 vias (octantes).

- Calculamos apenas 1/8 do círculo e espelhamos as coordenadas para os outros 7 quadrantes simultaneamente.

Algoritmo de Bresenham (Apenas Inteiros)

```
#include <GL/glut.h>
#include <cmath>

void algoritmoBresenham(int x1, int y1, int x2, int y2) {
    int dx = std::abs(x2 - x1);
    int dy = std::abs(y2 - y1);

    // Determina a direção do passo em X e Y (para suportar todos os quadrantes)
    int stepX = (x1 < x2) ? 1 : -1;
    int stepY = (y1 < y2) ? 1 : -1;

    // Variável de decisão inicial
    int erro = dx - dy;

    while (true) {
        desenhaPixel(x1, y1); // Usando a mesma função desenhaPixel da Aula 1

        if (x1 == x2 && y1 == y2) break; // Chegou no destino

        // Multiplicamos por 2 (pode ser feito com bitshift << 1)
        int erro2 = 2 * erro;

        // Ajusta a variável de decisão e a coordenada
        if (erro2 > -dy) {
            erro -= dy;
            x1 += stepX;
        }
        if (erro2 < dx) {
            erro += dx;
            y1 += stepY;
        }
    }
}
```

Vantagens de Bresenham

- Desempenho superior ao DDA.

- Precisão absoluta (sem erros de arredondamento acumulados).

O Problema do Preenchimento

- Como o computador sabe o que é "dentro" e "fora" de um polígono delineado por retas?

- Diferença entre preenchimento geométrico (matemático) e preenchimento de imagem (balde de tinta).

Algoritmo de Varredura (Scanline)

- Analisa a imagem linha por linha (como o feixe de um monitor CRT antigo).

- Encontra as interseções da linha de varredura com as arestas do polígono.

- Pinta os pixels entre os pares de interseção (liga/desliga).

Algoritmos por Semente (Flood Fill / Boundary Fill)

- Baseados na conectividade dos pixels (4-conexos ou 8-conexos).

- Flood Fill: Substitui uma cor de fundo específica por uma nova cor (ex: ferramenta balde de tinta clássica).

- Boundary Fill: Pinta até encontrar uma cor de borda específica.

Preenchimento (Flood Fill e Nativo)

```
#include <GL/glut.h>

// Estrutura simples para cor
struct Cor { float r, g, b; };

// Lê um pixel diretamente do Frame Buffer do OpenGL
Cor lePixel(int x, int y) {
    Cor cor;
    glReadPixels(x, y, 1, 1, GL_RGB, GL_FLOAT, &cor);
    return cor;
}

// Verifica se duas cores são iguais (simplificado)
bool coresIguais(Cor c1, Cor c2) {
    return (c1.r == c2.r && c1.g == c2.g && c1.b == c2.b);
}

// Função recursiva (Atenção: em áreas grandes, causa Stack Overflow!)
void floodFill(int x, int y, Cor corPreenchimento, Cor corFundo) {
    Cor corAtual = lePixel(x, y);

    if (coresIguais(corAtual, corFundo)) {
        // Pinta o pixel atual
        glColor3f(corPreenchimento.r, corPreenchimento.g, corPreenchimento.b);
        desenhaPixel(x, y);
        glFlush(); // Força o desenho imediato para vermos a animação (opcional)

        // Chamadas recursivas para as 4 direções
        floodFill(x + 1, y, corPreenchimento, corFundo); // Direita
        floodFill(x - 1, y, corPreenchimento, corFundo); // Esquerda
        floodFill(x, y + 1, corPreenchimento, corFundo); // Cima
        floodFill(x, y - 1, corPreenchimento, corFundo); // Baixo
    }
}
```

Como o OpenGL preenche

```
void preenchimentoNativo() {
    // Desenha um polígono preenchido automaticamente pela API
    glBegin(GL_POLYGON);
        glVertex2i(50, 50);
        glVertex2i(150, 50);
        glVertex2i(100, 150);
    glEnd();
}
```

Estruturas de Dados

- Recursividade no Flood Fill pode causar Stack Overflow em resoluções altas.

- Alternativas: Uso de Pilhas (Stacks) ou Filas (Queues) explícitas na memória.

Conceito de Recorte

- **Window**: A área do mundo contínuo que queremos visualizar.

- **Viewport**: A área da tela/janela matricial onde o gráfico será desenhado.

- Por que recortar? Economizar processamento na rasterização de partes invisíveis.

Recorte em 2D

```cpp
#include <iostream>

// Códigos de região (Bitmasks)
const int DENTRO   = 0; // 0000
const int ESQUERDA = 1; // 0001
const int DIREITA  = 2; // 0010
const int BAIXO    = 4; // 0100
const int CIMA     = 8; // 1000

// Limites da Viewport/Janela
const int X_MIN = 50, X_MAX = 200, Y_MIN = 50, Y_MAX = 200;

// Função para calcular o Outcode de um ponto (x,y)
int calculaOutcode(int x, int y) {
    int code = DENTRO;

    if (x < X_MIN)      code |= ESQUERDA; // Operação bitwise OR
    else if (x > X_MAX) code |= DIREITA;

    if (y < Y_MIN)      code |= BAIXO;
    else if (y > Y_MAX) code |= CIMA;

    return code;
}

// Exemplo didático para rodar no console ou adaptar pro OpenGL
void verificaReta(int x1, int y1, int x2, int y2) {
    int outcode1 = calculaOutcode(x1, y1);
    int outcode2 = calculaOutcode(x2, y2);

    if ((outcode1 == 0) && (outcode2 == 0)) {
        std::cout << "Reta " << x1 << "," << y1 << " ate " << x2 << "," << y2
                  << " -> Aceite Trivial (Totalmente Dentro)\n";
    }
    else if (outcode1 & outcode2) { // Operação bitwise AND
        std::cout << "Reta " << x1 << "," << y1 << " ate " << x2 << "," << y2
                  << " -> Rejeite Trivial (Totalmente Fora)\n";
    }
    else {
        std::cout << "Reta " << x1 << "," << y1 << " ate " << x2 << "," << y2
                  << " -> Precisa calcular intersecao (Clipping necessario)\n";
    }
}
```

Algoritmo de Cohen-Sutherland para Retas

- Divide o espaço ao redor da janela de recorte em 9 regiões.

- Cada região recebe um código binário de 4 bits (Outcode): Topo, Fundo, Direita, Esquerda. (ex: 1001 significa Topo e Esquerda).

A Lógica do Cohen-Sutherland

- A janela central (área visível) tem o código 0000.

- Aceite Trivial: Se ambos os extremos da reta são 0000 (reta 100% dentro).

- Rejeite Trivial: Se o AND lógico (&) dos códigos dos extremos for diferente de 0000 (reta 100% fora no mesmo lado).

- Interseção: Se falhar nos dois acima, calcula-se a interseção da reta com as bordas da janela para parti-la e testar de novo.